Image Transformations

CST 205

Review: Grayscale (“Luminosity” Method)

```
def grayscale(p):
    lumi = int( p[0]*0.299 + p[1]*0.587 + p[2]*0.114 )
    return (lumi,) * 3

def gray_list(pic):
    gray_list = [grayscale(p) for p in pic.getdata()]
    return gray_list
```

One more filter

Sepia: a brown pigment obtained from the inklike secretion of various cuttlefish and used with brush or pen in drawing.

Sepia Code

```
def sepia(p):
    # tint shadows
    if p[0] < 63:
        r,g,b = int(p[0] * 1.1), p[1], int(p[2] * 0.9)

    # tint midtones
    elif p[0] > 62 and p[0] < 192:
        r,g,b = int(p[0] * 1.15), p[1], int(p[2] * 0.85)

    # tint highlights
    else:
        r = int(p[0] * 1.08)
        g,b = p[1], int(p[2] * 0.5)

    return r, g, b
```

Moving pixels

- We need to keep track of where we're getting pixels from and where we're putting the pixels
  - *Source* index variables: where we're getting the pixels from
  - *Target* index variables: where we're putting the pixels

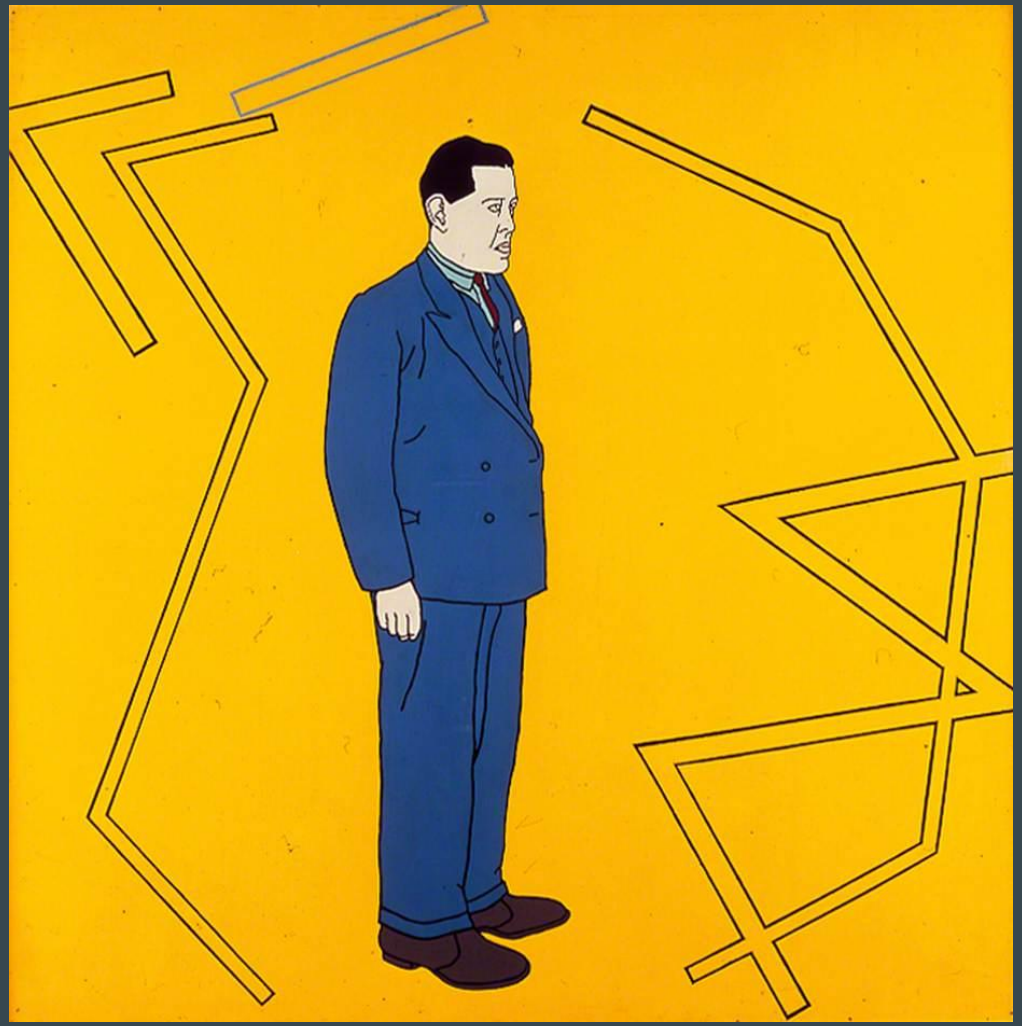
Note: We are not moving the *actual* pixels, but rather just replicating the colors.

Uses

- Collages
- Cropping
- Scaling
- and more!

Review: Empty images in Pillow

```
from PIL import Image
im = Image.new('RGB', (640,480), 'salmon')
im.show()
```



"Portrait of Juan Gris" by
Patrick Caulfield

Copying an image to a blank canvas

```
def copy_image(your_image):
    my_src = Image.open(your_image)
    w,h = my_src.width, my_src.height
    my_trgt = Image.new('RGB', (w,h))

    target_x = 0
    for source_x in range(w):
        target_y = 0
        for source_y in range(h):
            p = my_src.getpixel((source_x, source_y))
            my_trgt.putpixel((target_x, target_y), p)
            target_y += 1
        target_x += 1

    my_trgt.show()
```

Transformations

Making small changes to this basic idea can result in a variety of transformations

- Change `target_x` and `target_y` to copy wherever you want
- **Cropping:** Change the `source_x` and `source_y` range and you can copy only part of the image
- **Rotating:** Swap `target_x` and `target_y` and you copy sideways
- **Scaling:** Change the increment on `source_x` and `source_y` and you either grow or shrink the image

Shift and copy

```
my_src = Image.open('juan_gris.jpg')

new_w, new_h = my_src.width + 200, my_src.height + 200
my_trgt = Image.new('RGB', (new_w, new_h), 'salmon')

target_x = 100
for source_x in range(my_src.width):
   target_y = 100
   for source_y in range(my_src.height):
       pixel = my_src.getpixel((source_x, source_y))
       my_trgt.putpixel((target_x, target_y), pixel)
       target_y += 1
   target_x += 1

my_trgt.show()
```

Scaling

- When we *just* copy, we sample every pixel

- If we want a smaller copy, we skip some pixels
  - We sample fewer pixels

- If we want a larger copy, we duplicate some pixels
  - We oversample some pixels

Sample fewer pixels

```
my_src = Image.open('juan_gris.jpg')
w, h = my_src.width//2, my_src.height//2
my_trgt = Image.new('RGB', (w, h))

target_x = 0
for source_x in range(0, my_src.width, 2):
   target_y = 0
   for source_y in range(0, my_src.height, 2):
       p = my_src.getpixel((source_x, source_y))
       my_trgt.putpixel((target_x, target_y), p)
       target_y += 1
   target_x += 1

my_trgt.show()
```

Quick NumPy aside

- **NumPy** is a popular Python package for performing scientific computing.
- Can install it using:
  `pip install numpy`
  (with your virtual environment activated)

```
import numpy as np
my_list = range(1,6)

dupe_list = np.repeat(my_list, 2)
print(dupe_list)

# printable representation of an object
print(repr(dupe_list))

print(dupe_list[4])
print(type(dupe_list[4]))
```

Code for scaling-up

```
my_src = Image.open('juan_gris.jpg')
mf = 2
w, h = my_src.width*mf, my_src.height*mf

my_trgt = Image.new('RGB', (w, h))
target_x = 0
for source_x in np.repeat(range(my_src.width), mf):
   target_y = 0
   for source_y in np.repeat(range(my_src.height), mf):
       p = my_src.getpixel((int(source_x), int(source_y)))
       my_trgt.putpixel((target_x, target_y), p)
       target_y += 1
   target_x += 1

my_trgt.show()
```

Blending Pictures

- Instead of copying from source to target, we can combine the source and target to create a new image
- Simple technique
  - Average the red, green, and blue from the source and target

Let's put Juan Gris on the beach!

Blending (code)

```
my_src = Image.open('juan_gris.jpg')
my_trgt = Image.open('beach.jpg')

x_shift = my_trgt.width//2 - 1400
y_shift = my_trgt.height//2 + 400

source_x = 0
for x in range(x_shift, x_shift + my_src.width):
   source_y = 0
   for y in range(y_shift, y_shift + my_src.height):
       r_s, g_s, b_s = my_src.getpixel((source_x, source_y))
       r_t, g_t, b_t = my_trgt.getpixel((x, y))
       blend = ( (r_s+r_t)//2, (g_s+g_t//2), (b_s+b_t)//2 )
       my_trgt.putpixel((x,y), blend)
       source_y += 1
   source_x += 1
my_trgt.show()
```

Chroma key compositing

- Think of the weather person on the news
- Pose in front of a blue or green screen
- Swap all blue or green for the background

Chroma key code

```
def chromakey(src, bg, refp):
    for x in range(src.width):
        for y in range(src.height):
            cur_pixel = src.getpixel((x,y))

            if color_distance(cur_pixel, refp) < 150:
                src.putpixel((x,y), bg.getpixel((x,y)))

    return src
```